

目标函数改造设计

1. 核心定义与符号表

- $G = (V, E)$: 全局语义标签图, 包含所有语义标签 (tag) 节点。
- $L_q \subseteq V$: 测试问题 (Query) q 提取出的语义标签集合。
- $D = \{d_i\}_{i=1}^N$: 候选示例库 (Demonstration Pool), $N = |D|$ 。
- $df(l)$: 标签 l 在候选库中出现的文档频次 (Document Frequency, 按样本计 0/1)。
- $\mathcal{C}$: 难度标签集合, 例如 $\{Easy, Medium, Hard\}$, 或更细粒度 bins / KDE 连续建模 (见“难度标签设计”)。
- v_i : 样本 i 的特征向量, 由 **语义部分** v_i^{sem} 和 **难度部分** v_i^{diff} 拼接而成:

$$v_i = v_i^{sem} \parallel v_i^{diff}$$

- s_i : 样本 i 的信息密度分数 (质量/长度), 用于语义信息与 (bins 平滑方案中的) 难度增量权重。
- A : 语义图的传播矩阵 (用于模糊匹配 / 语义扩散)。
- $c_i \in \mathbb{R}$: 样本 i 的难度分数 (complexity score)。
- $\hat{p}_S(c)$: 集合 S 的难度分数概率密度函数 (pdf) 估计。
- $K_\sigma(\cdot)$: KDE 核函数 (高斯核), σ 为带宽 (bandwidth)。
- $w(c) \geq 0$: 难度轴位置 c 的权重函数 (默认 $w(c) \equiv 1$; 可选用于强调某段难度)。

2. 构建二维目标权重向量 $w(q)$

我们需要为系统中的每一个标签节点 j 计算一个针对当前 Query 的需求权重。这个权重由 **相关性** 和 **特异性** 的乘积决定, 充当了“注意力门控”的角色。

对于难度标签, 我们赋予固定的基础权重, 以确保难度分布始终被优化。

A. 第一维: 相关性 (Relevance, $r_j(q)$)

相关性 $R_j(q)$ 采用三档离散:

令

$$dist(l_j, L_q) = \min_{l \in L_q} dist(l_j, l)$$

为标签 l_j 到 Query 标签集合的最短 hop 距离，则

$$R_j(q) = \begin{cases} 1, & dist(l_j, L_q) = 0 \\ 0.5, & dist(l_j, L_q) = 1 \\ 0, & dist(l_j, L_q) \geq 2 \end{cases}$$

B. 第二维：特异性 (Specificity, S_j)

基于候选库中的文档频率倒排 (IDF 思想)，衡量标签的稀缺度。越稀有的标签信息量越大：

$$S_j \triangleq idf(l_j) = \log \frac{N + 1}{df(l_j) + 1}$$

(注：分母加 1 是为了平滑处理；并且由于 $0 \leq df(l_j) \leq N$ ，所以 $\frac{N+1}{df(l_j)+1} \geq 1$ ，从而 $idf \geq 0$ ，避免出现负权重。)

3. 目标函数设计 (The Objective Function)

这是改造的核心。我们将 **语义匹配** 和 **难度调控** 统一在一个非线性函数 $\Phi(\cdot)$ 中，利用其凹函数特性 (Concavity) 实现“边际收益递减”。

A. 统一信息向量构造

对于样本集合 S ，我们定义其在所有维度上的 **累积加权信息量向量**：

$$x(S) = \sum_{i \in S} \hat{e}_i$$

其中每个样本的“统一信息向量”定义为 (语义扩散 + 难度独立计数)：

$$\hat{e}_i \triangleq (A \cdot e_i^{sem}) \parallel (\lambda \cdot e_i^{diff})$$

- $e_i^{sem} = s_i \cdot v_i^{sem}$ ：语义部分 (质量/长度加权)。
- $A \cdot e_i^{sem}$ ：语义标签经过图传播矩阵 A 进行平滑，捕捉潜在语义。
- e_i^{diff} ：难度部分 (不经过矩阵 A ，保持独立计数/建模)。
- λ ：调节难度重要性的超参数。
- $\parallel$ ：向量拼接操作。

B. 全局最大化目标

我们寻找一个子集 S (满足预算约束, 如示例条数 K 或 token budget) , 使得以下目标函数最大化:

$$F(S; q) = \Phi(x(S); q) = \sum_j w_j(q) \cdot \phi(x_j(S))$$

其中:

- $\phi(\cdot)$: 必须是单调递增的凹函数, 例如 $\phi(x) = \log(1 + x)$ 或 $\phi(x) = x^{0.8}$ 。

难度标签设计

1) 简单离散

把 complexity 得分简单划分三档, 作为三种标签:

$$\mathcal{C} = \{Easy, Medium, Hard\}$$

并令 v_i^{diff} 为 one-hot (或计数) 向量。

2) 更细粒度离散 + 平滑加成 (binning + smoothing)

1.1 定义难度轴的 bins

假设 complexity 原始分数范围是 $[c_{\min}, c_{\max}]$ 。

设 bin 宽度 $h = 0.1$, bin 中心:

$$b_k = c_{\min} + k \cdot h, \quad k = 0, 1, \dots, K - 1$$

难度维度数:

$$K = \left\lfloor \frac{c_{\max} - c_{\min}}{h} \right\rfloor + 1$$

1.2 核平滑：把一个样本的难度“摊”到邻近 bins

样本 i 的难度分数是 c_i 。给每个 bin 的贡献：

$$g_k(c_i) = \exp\left(-\frac{(c_i - b_k)^2}{2\sigma^2}\right)$$

然后做截断（只保留附近窗口）+ 归一化：

- 只取

$$|c_i - b_k| \leq r$$

的 bins（比如 $r = 0.3$ ，对应最多 7 个 bin，计算很轻）

- 归一化到和为 1：

$$\tilde{g}_k(c_i) = \frac{g_k(c_i)}{\sum_{k' \in \text{near}(c_i)} g_{k'}(c_i)}, \quad \text{near}(c_i) = \{k : |c_i - b_k| \leq r\}$$

最后把难度增量定义为：

$$\Delta_{diff,k}(i) = s_i \cdot \tilde{g}_k(c_i)$$

- σ 小：更尖锐（几乎只在 3.3 附近）
- σ 大：更平滑（更宽）

1.3 对 MIG-ICL 目标函数怎么接

（该式为 **难度 bins 部分** 的边际增益；语义增量可与其相加或用统一 $\text{supp}(\Delta x(i))$ 形式写在一起。）

$$\Delta F_{diff}(i | S) = \sum_{k \in \text{near}(c_i)} w_k [\phi(x_k + \Delta_{diff,k}(i)) - \phi(x_k)]$$

因为 $\text{near}(c_i)$ 的 bins 很少，所以每次增量计算依然快。

3) 连续密度函数 (KDE) + 凹函数 (continuous density + concave utility)

1.1 定义难度的概率密度函数 (KDE)

假设每个样本 i 有难度分数 (complexity) $c_i \in \mathbb{R}$ 。

对已选集合 S ，用 KDE 拟合其难度 pdf:

$$\rho_S(c) = \sum_{i \in S} K_\sigma(c - c_i)$$

其中核函数取归一化高斯核:

$$K_\sigma(\Delta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\Delta^2}{2\sigma^2}\right)$$

因此 $\int \hat{p}_S(c) dc = 1$ 。

σ 控制平滑程度:

- σ 小: 密度是很多尖峰 (更“精确”但不平滑)
- σ 大: 密度更平滑 (更“泛化”)

1.2 用凹函数把 pdf 变成“覆盖收益” (边际递减)

选一个单调递增的凹函数 $\phi(\cdot)$ 。

定义难度覆盖目标为对整个难度范围的积分:

$$F_{diff}(S) = \int_{c_{\min}}^{c_{\max}} \phi(\hat{p}_S(c)) dc$$

当 ϕ 为凹函数时, ϕ 带来边际递减: 某段难度密度已经很高时, 再往同段堆样本, 收益变小 $\rightarrow$ 促进难度覆盖更均衡。

1.3 贪心选择时的边际增益 (连续形式, pdf 版本)

当前集合 S 的密度是 $\rho_S(c)$ 。加入候选样本 i 后密度变为:

$$\rho_{S \cup \{i\}}(c) = \rho_S(c) + K_\sigma(c - c_i)$$

因此边际增益为:

$$\Delta F_{diff}(i | S) = \int_{c_{\min}}^{c_{\max}} \left[\phi(\rho_S(c) + K_\sigma(c - c_i)) - \phi(\rho_S(c)) \right] dc$$

5. 贪婪选择算法流程

输入：候选库 D , Query 标签 L_q , 目标数量 K 。

输出：选定示例集合 S 。

1. 初始化：

- 计算目标权重向量 $w(q)$ 。
- 初始化已选集合 $S \leftarrow \emptyset$, 当前累积信息向量 $x \leftarrow 0$ 。

2. 迭代选择 ($k = 1$ to K) :

对于每个候选样本 $d_i \in D \setminus S$:

- 计算该样本的增量向量：

$$\Delta x(i) = \hat{e}_i$$

- 计算边际收益 (Marginal Gain):

$$\Delta F(i | S) = \sum_{j \in \text{supp}(\Delta x(i))} w_j(q) \left[\phi(x_j + \Delta x_j(i)) - \phi(x_j) \right]$$

选择 $\Delta F(i | S)$ 最大的样本 i^* 。

更新集合: $S \leftarrow S \cup \{i^*\}$ 。

更新状态: $x \leftarrow x + \Delta x(i^*)$ 。

3. 返回集合 S 。

评分函数

修正后的 DEITA

将 DEITA 的公式中的 **信息密度项** 定义为 (示例写法) :

$$s_i = \frac{\text{quality}_i}{\text{len}_i^\beta} \quad \text{或} \quad s_i = \text{quality}_i \cdot e^{-\beta \text{len}_i}$$

逻辑：引入长度惩罚（Length Penalty）。 β 是超参数（例如 0.5 或 1.0）。这样会惩罚那些“又臭又长”的例子，奖励“短小精悍”的高质量例子。

适配的任务类型

处理混合任务：比如包括[代码, 数学, 翻译, 摘要]等的任务集合

单类型任务：复杂推理与数学问题、代码生成、风格化或受限文本生成、复杂的分类任务

暂定：Python 代码生成（Code Generation）

候选池 (Demonstration Pool)：Open Hermes 中 python 相关数据

评测集 (Test Query)：HumanEval

HumanEval 没有训练集，它设计初衷就是作为“不可见”的测试集。

逻辑：用候选池的题目教会模型“如何写 Python 函数”，然后让它解决 HumanEval 的问题。

补充实验

示例排序问题

1. 利用公式，单独计算选出来的示例对于测试示例标签的覆盖度（即只选这个示例时的得分），最相关的示例靠近测试问题
2. 基于示例标签距离测试示例标签的距离远近排序
3. 直接利用 scorer 分数
4. 基于 MIG 选择顺序的相反顺序（最相关的排在最后，补充示例排在最前）

长度惩罚

SFT 数据集通常不怎么在乎长度，但 ICL 的 Context Window 是寸土寸金的。

加长度作为惩罚项（做 AB 组）（加减除）

多步扩散

语义扩散时可做多步扩散：k 步扩散、多步加权和、限定在 Query 邻域的多步扩散

- k 步扩散：

$$\hat{e}^{sem} = A^k e^{sem}$$

- 多步加权和（PPR/RWR 截断）：

$$\hat{e}^{sem} = \sum_{t=0}^{K_d} \beta_t A^t e^{sem} \quad (\beta_t \text{ 可取几何衰减})$$

- 限定在 Query 邻域的多步扩散：先构造 Query 邻域子图，再在子图上做上述扩散以防漂移。

评测方案（简述）

评测数据

1. 候选示例池： data/icl/openhermes_python_related.jsonl
2. 评测查询集： data/icl/humaneval_eval_tagged.jsonl
3. MIG 选例映射： data/icl/mappings/humaneval_to_demos.jsonl
4. 评测标准：HumanEval 每题自带 tests.test，按 pass@1 / pass@k 统计。

实验分组

1. Zero-shot：不加示例，直接解题。
2. Random-ICL：随机抽取 k=8 条示例。
3. Similarity-ICL：按相似度检索 k=8 条示例。
4. MIG-ICL：使用 ordered_demo_ids 的顺序组装 k=8 条示例。

可选消融组：

1. 去掉长度惩罚（lambda_len=0）
2. 去掉冗余惩罚（lambda_red=0）
3. 去掉 IDF 权重（仅 relevance）

评测流程

1. 按实验组构建 prompt (统一 system 指令 + 示例 + query) 。
2. 调用同一 API 模型生成代码答案并保存原始输出。
3. 解析代码 completion, 执行 HumanEval 测试用例。
4. 汇总各组 pass@1 / pass@k、平均 token、平均延迟和失败类型。

结果产物

1. 推理输出: data/icl/eval_outputs/<run_name>/*.jsonl
2. 指标汇总: data/icl/eval_metrics/<run_name>/summary.json
3. 对比报告: docs/评测方案.md 中定义的表头格式落盘为 markdown/csv 。